

Atomicity vs Critical Section

Ensuring each thread completes before another can complete some code

Atomic example

```
#pragma omp parallel for
for (i = 0; i < REPS; i++) {
    #pragma omp atomic
    balance += 1.0;
}
```

Critical section example

```
#pragma omp parallel for
for (i = 0; i < REPS; i++) {
    #pragma omp critical
    {
        balance += 1.0;
    }
}
```

Race conditions

balance++; // balance has permanent storage in RAM

The above statement can be compiled into, at least three instructions.

1. Fetching *balance* value (into a register)
2. Incrementing *balance* value
3. Storing the updated value (back to memory)

LOAD src register

ADD 1 register

STORE register dest_mem

One of several scenarios

Thread 0

- 1.Fetching *balance* value (into a register)
- 2.Incrementing *balance* value
- 3.Storing the updated value (back to memory)

Thread 1

- 1.Fetching *balance* value (into a register)
- 2.Incrementing *balance* value
- 3.Storing the updated value (back to memory)

XADD combines these operations

From this site: <https://sites.google.com/site/microprocessorsbits/arithmetic-instructions/xadd>

EXCHANGE AND ADD (486+)

DESCRIPTION: The XADD instruction adds the source to the destination and stores the sum in the destination, as with any addition. The difference is that after the addition takes place, the original value of the destination is copied into source operand.

USAGE: XADD dest, src

Replaces this assembly code:

LOAD src register1
LOAD dest register2
ADD register1 register2
STORE register1 dest

*#pragma omp atomic
balance++;*

can compile to this:

LOAD 1 R8 # each thread has its own register

#each thread does an atomic instruction:

LOCK XADD R8 src

src is shared memory location

Few Atomic instructions

There are only a few operations that have atomic instructions

Depends on the CPU architecture/instruction set

Add

Subtract

Multiply

Divide

Bit-level operations (&, |, ^, >>, <<)

Applies only to a single line of code following the omp atomic pragma

Critical sections

Place around a block of lines of code

Signifies that the code must all be executed before that next thread gets to execute it.

Causes threads to wait until one inside its critical section completes

Does not compile into CPU atomic instructions